



Bahria University, Islamabad

Department of Software Engineering

Data Structures and Algorithms (Spring-2026)

Lab Instructor: Engr. Muhammad Faisal Zia

Student:

Enrollment:

Lab Journal: 02

Date:

Task No:	Task Wise Marks		Documentation Marks		Total Marks (25)
	Assigned	Obtained	Assigned	Obtained	
1					
2					
3					
4					
5					

Comments:

Signature

Lab # 02

Implementation of Stack

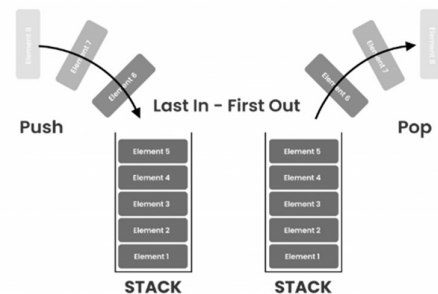
Objectives

The objective of this lab is to understand and implement the Stack abstract data type (ADT) in C++. By the end of this lab, you should be able to:

- Understand the Stack ADT and its operations
- Implement a stack using static arrays
- Implement a stack using dynamic arrays with class templates
- Apply stack operations to solve practical problems
- Work with file I/O and arrays of objects

Introduction

A stack is a fundamental data structure that follows the Last-In-First-Out (LIFO) principle. This means that the last element added to the stack will be the first one to be removed. Stacks are widely used in computer science for various applications including expression evaluation, function call management, undo operations, and syntax parsing.



Stack ADT Operations

- **Core Stack Operations**

The Stack Abstract Data Type (ADT) typically includes the following operations:

1. **Initialize** - Sets the stack to an empty state
2. **IsEmpty** - Determines whether the stack is currently empty
3. **IsFull** - Determines whether the stack is currently full (for array implementations)
4. **Push (ItemType newItem)** - Adds newItem to the top of the stack
5. **Pop (ItemType& item)** - Removes the item at the top of the stack and returns it in item

Implementation of Stack Using Static Array

- **Header File (stack.h)**

```
// SPECIFICATION FILE (stack.h)
#pragma once
#define MAX_ITEMS 100
typedef int ItemType;

class Stack {
public:
    Stack(); // Default constructor
    int IsEmpty() const; // Check if stack is empty
```

```

    int IsFull() const;           // Check if stack is full
    void Push(ItemType newItem); // Add item to top of stack
    void Pop(ItemType& item);     // Remove item from top of stack

private:
    int top;                      // Index of top element
    ItemType items[MAX_ITEMS];   // Array to store stack elements
};

```

- Implementation File (stack.cpp)

```

// IMPLEMENTATION FILE (stack.cpp)
#include "stack.h"
#include <iostream>
#include <cstdlib>
#include <stdexcept>
using namespace std;

// Default Constructor
Stack::Stack() : top(0) { // Initialize items array to zero
}

// Check if stack is empty
int Stack::IsEmpty() const {
    return top == 0;
}

// Check if stack is full
int Stack::IsFull() const {
    return top == MAX_ITEMS;
}

// Push operation - Add item to top of stack
void Stack::Push(ItemType newItem) {
    if (IsFull()) {
        throw std::overflow_error("Stack::Push on full stack");
    }
    items[top++] = newItem;
}

// Pop operation - Remove item from top of stack
void Stack::Pop(ItemType& item) {
    if (IsEmpty()) {
        throw std::underflow_error("Stack::Pop on empty stack");
    }
}

```

```

        item = items[--top];
    }

```

- **Driver Program (driver.cpp)**

```

// DRIVER FILE (driver.cpp)
#include <iostream>
#include "stack.h"
using namespace std;

int main() {
    Stack s;    // Create stack object
    int item;   // Variable to store popped item

    // Push 20 items onto the stack
    cout << "Pushing items onto stack:" << endl;
    for (int i = 0; i < 20; i++) {
        s.Push(i);
        cout << "Pushed: " << i << endl;
    }

    cout << "\nPopping items from stack (in reverse order):" <<
endl;
    // Pop 20 items from the stack
    for (int i = 0; i < 20; i++) {
        s.Pop(item);
        cout << "Popped: " << item << endl;
    }

    return 0;
}

```

-

Dynamic Implementation of Stack

- **Stack Using Class Template and Dynamic Array**

Class templates allow us to create generic classes that can work with different data types. This is particularly useful for implementing data structures like stacks.

- **Class Template Definition (stack.h)**

```

#pragma once
// CLASS TEMPLATE DEFINITION FILE (stack.h)
#include <iostream>
#include <cstdlib>
using namespace std;

```

```

template<class ItemType>
class Stack {
public:
    Stack();                // Default constructor
    Stack(int max);         // Parameterized constructor
    ~Stack();               // Destructor
    int IsEmpty() const;    // Check if stack is empty
    int IsFull() const;     // Check if stack is full
    void Push(ItemType newItem); // Add item to top of stack
    void Pop(ItemType& item);  // Remove item from top of stack

private:
    int top;                // Index of top element
    int maxStack;           // Maximum stack size
    ItemType* items;        // Dynamic array for stack elements
};

#endif

```

- **Class Template Implementation (stack.cpp)**

```

// CLASS TEMPLATE IMPLEMENTATION FILE (stack.cpp)
#include "stack.h"

// Default Constructor
template<class ItemType>
Stack<ItemType>::Stack() {
    maxStack = 500;
    top = -1;
    items = new ItemType[500]; // Dynamically allocate array
}

// Parameterized Constructor
template<class ItemType>
Stack<ItemType>::Stack(int max) {
    maxStack = max;
    top = -1;
    items = new ItemType[max]; // Dynamically allocate array
}

// Destructor
template<class ItemType>
Stack<ItemType>::~~Stack() {
    delete[] items; // Deallocate array
}

// Check if stack is empty
template<class ItemType>

```

```

int Stack<ItemType>::IsEmpty() const {
    return (top == -1);
}

// Check if stack is full
template<class ItemType>
int Stack<ItemType>::IsFull() const {
    return (top == maxStack - 1);
}

// Push operation
template<class ItemType>
void Stack<ItemType>::Push(ItemType newItem) {
    if (IsFull()) {
        cout << "Stack Overflow" << endl;
        exit(1);
    }
    top++;
    items[top] = newItem;
}

// Pop operation
template<class ItemType>
void Stack<ItemType>::Pop(ItemType& item) {
    if (IsEmpty()) {
        cout << "Stack Underflow" << endl;
        exit(1);
    }
    item = items[top];
    top--;
}

```

- Driver Program with Templates (driver_template.cpp)

```

// DRIVER PROGRAM USING CLASS TEMPLATE
#include <iostream>
#include "stack.cpp"
using namespace std;

int main() {
    // Create stacks of different types
    Stack<int> IntStack(10);    // Integer stack with max size 10
    Stack<float> FloatStack(5); // Float stack with max size 5
    Stack<char> CharStack(8);  // Character stack with max size 8

    int intData;
    float floatVal;
    char charVal;
}

```

```

// Test integer stack
cout << "Testing Integer Stack:" << endl;
for (int i = 1; i <= 5; i++) {
    IntStack.Push(i * 10);
    cout << "Pushed: " << i * 10 << endl;
}

cout << "\nPopping from Integer Stack:" << endl;
while (!IntStack.IsEmpty()) {
    IntStack.Pop(intData);
    cout << "Popped: " << intData << endl;
}

// Test float stack
cout << "\nTesting Float Stack:" << endl;
FloatStack.Push(3.1415927);
FloatStack.Push(2.7182818);

cout << "Popping from Float Stack:" << endl;
FloatStack.Pop(floatVal);
cout << "Popped: " << floatVal << endl;
FloatStack.Pop(floatVal);
cout << "Popped: " << floatVal << endl;

// Test character stack (string reversal example)
cout << "\nTesting Character Stack (String Reversal):" << endl;
char str[] = "STACK";
cout << "Original string: " << str << endl;

// Push characters onto stack
for (int i = 0; str[i] != '\0'; i++) {
    CharStack.Push(str[i]);
}

// Pop characters to reverse string
cout << "Reversed string: ";
while (!CharStack.IsEmpty()) {
    CharStack.Pop(charVal);
    cout << charVal;
}
cout << endl;

return 0;
}

```

Stack Applications

- **Expression Evaluation:** Stacks are commonly used to evaluate arithmetic expressions in prefix, infix, or postfix notation.
- **Function Call Management:** The call stack in programming languages uses the stack data structure to manage function calls and returns.
- **Undo Operations:** Many applications use stacks to implement undo functionality by pushing each state onto a stack.
- **Syntax Parsing:** Compilers use stacks to check for balanced parentheses and braces in source code.

Additional Stack Operations

- **Peek/Top Operation**

```
// Add this to the Stack class definition
ItemType Peek() const;

// Implementation
template<class ItemType>
ItemType Stack<ItemType>::Peek() const {
    if (IsEmpty()) {
        cout << "Stack is empty" << endl;
        exit(1);
    }
    return items[top];
}
```

- **Stack Size Operation**

```
// Add this to the Stack class definition
int Size() const;

// Implementation
template<class ItemType>
int Stack<ItemType>::Size() const {
    return top + 1;
}
```

Conclusion

The stack is a fundamental data structure with numerous applications in computer science. In this lab, we have explored two implementations: using static arrays (fixed size) and dynamic arrays with class templates (flexible size). The static implementation is simpler but less flexible, while the template-based dynamic implementation is more versatile and reusable. Understanding both approaches provides a solid foundation for working with more complex data structures.

Lab tasks

Task 01:

- a. Write a complete program to implement a stack using dynamic memory allocation. Your implementation should include:
 - A template class `DynamicStack` that dynamically allocates memory
 - Constructors (default and parameterized) and destructor
 - All basic stack operations (Push, Pop, IsEmpty, IsFull)
 - A driver program that demonstrates all operations
- b. Write a program that uses stack operations to find the maximum and minimum values in a sequence of numbers.

Task 02:

You will write a student grades "database" program. It will read data of students from a file and will let the user perform various operations on the data. You will have to store the student data in an array of objects.

The input file will look like:

4	// number of students
3	// number of grades (per student)
Hassan Khan 99 87 90	// Student name grade grade ... grade
Sara Nazir 90 98 99	// Student name grade grade ... grade
Ali Zaidi 55 43 0	// Student name grade grade ... grade
Raza Ahmad 100 100 100	// Student name grade grade ... grade

Data structure:

You will store all the information in an array of "student" objects. You may use the following class definition:

Class Definition:

```
class Student {
private:
    char name[30];
    int lab[10];
    float average;
    int gradeCount;

public:
    .
    .
    .
};
```

NOTE:

Your program should work as follows:

- Ask the user for the filename and open the file.
- Read in the input from the file and store it in the student array.
- Compute and store an average for every student.
- Go into a menu loop giving the user the following options:
 1. Print all usernames, all grades, and averages.
 2. Find a student and print his/her information.
 3. Quit.
- For option 1 the user doesn't have to give you any extra information.
- For option 2, finding a student, your program must ask the user for the name of the student he/she wishes to find; read in the name; perform a sequential search for that name; and if found, print all that student's info.